SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Computer Simulation Foundations

Audi Development Camp 2026

Rudolf Krecht

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER

Agenda

1	**Why simulate robots?**	Motivation and the reality gap
2	**Anatomy of a robot simulator**	World → model → physics → sensors → actuators
3	**The simulator landscape**	Gazebo, Webots, CoppeliaSim, Isaac Sim, MuJoCo
4	**MuJoCo in depth**	MJCF, contacts, Python API
5	**Legged & humanoid robots**	Quadrupeds → bipeds → humanoids
6	**Validation, takeaways, Q&A**	What to build next

What is a robot simulator?

A **robot simulator** is a computer program that pretends to be the physical world. It represents a robot as a mathematical model, applies the laws of physics to that model in software, and lets you interact with the result **as if** the robot were real. It might provide realistic appearance.

→ Why does the field prefer a simulator over the real robot?

→ Where does a simulator lie about reality, and how much?

→ When is simulation the wrong tool for the job?

Why do we use simulators? - Cost

Real hardware

- A research-grade quadruped like **Boston Dynamics Spot** or **Unitree Go2** is a **€ 40 000 – 100 000** commitment.
- A humanoid research platform easily crosses **€ 200 000**. Each broken motor is a purchase order and a two-week wait for parts.

Simulator

- The same robot, in **MuJoCo** or **Gazebo**, runs on a laptop.
- Cost per broken robot: **€ 0**.
- Cost per additional robot: the seconds it takes to duplicate an XML file. You can run **2 000** of them in parallel on a single GPU.

Practical consequence: the sim is where you make mistakes. The real robot is where you confirm the mistakes are already fixed.

Why do we use simulators? – Risk factor

Early controller code contains bugs. A bug on a small 3-kg robot arm bruises a finger. The same bug on a **200 kg humanoid** becomes a headline. Simulation absorbs the entire risk of the debugging phase.

Three risks the sim eliminates

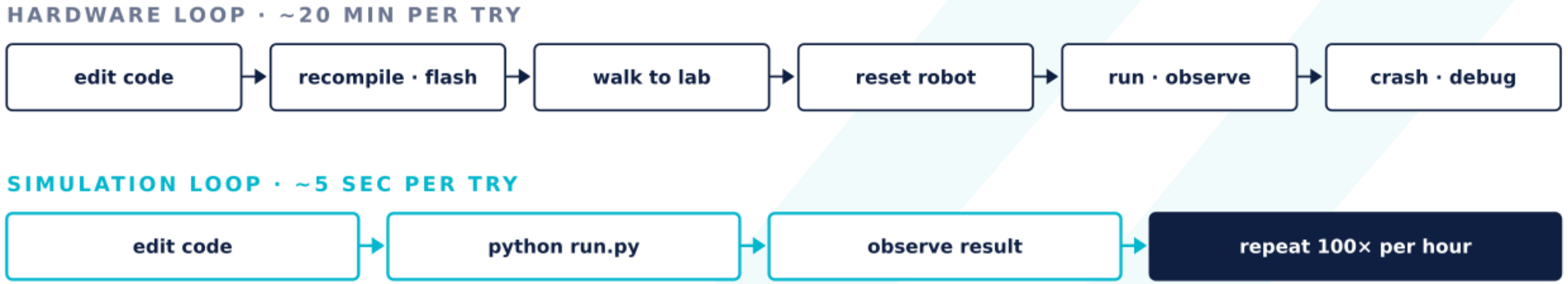
RISK 1
Injury to humans
A runaway robot in the lab can strike operators. In sim, worst case is a stack trace.

RISK 2
Damage to the robot
Reduction gearboxes shatter on impact. Sim allows unlimited crashes.

RISK 3
Damage to the environment
A drone can crash into equipment or through a wall. Sim walls do not break.

Why do we use simulators? – Iterative development

Modern robotics is fundamentally an **iterative** discipline — you rarely get a controller right on the first try. The question is not *whether* you will iterate, but how fast each iteration takes.



Why do we use simulators? – Repeatability

The problem with real experiments

Two runs of the same experiment on the same hardware never come out identical.

- Motor temperature drifts
- Floor friction depends on humidity
- Gears wear
- Initial pose is never truly identical

Determinism in simulation

A simulator with a fixed random seed and a fixed initial state produces the **exact same** trajectory on every run, byte for byte.

This is what makes it possible to:

- Compare two controllers *fairly*
- Reproduce a bug on demand
- Publish results others can verify

Why do we use simulators? – Scalability

Simulation is what makes learning-based control possible

Modern reinforcement-learning controllers — the ones behind the legged locomotion of existing humanoids — require the robot to try, fail, and learn **tens of millions** of episodes before the policy converges.

On real hardware

≈ 300 years

to collect the training data ANYmal policies use — assuming continuous 24/7 operation with no maintenance.

In parallel simulation

≈ 4 hours

on a single workstation with 4 000 parallel robots (Isaac Sim / MuJoCo XLA).

Without simulation, learning-based robotics simply does not happen.

Simulation is always an approximation

> reality = simulation + reality gap

Every simulator makes **deliberate simplifications** — the world is smooth where the real one is textured, motors are ideal where real motors have hysteresis, sensors are noise-free where real sensors drift. That trade-off is what makes simulation fast and cheap. It is also what makes the "reality gap" the central concern of applied robotics.

> **Rule of thumb —** if a result only appears in simulation and disappears on the robot, either the sim is wrong *or* the controller is exploiting a sim artifact. Both mean: it needs to be fixed.

Simulation is always an approximation

> reality = simulation + reality gap

Every simulator makes **deliberate simplifications** — the world is smooth where the real one is textured, motors are ideal where real motors have hysteresis, sensors are noise-free where real sensors drift. That trade-off is what makes simulation fast and cheap. It is also what makes the "reality gap" the central concern of applied robotics.

> **Rule of thumb —** if a result only appears in simulation and disappears on the robot, either the sim is wrong *or* the controller is exploiting a sim artifact. Both mean: it needs to be fixed.

Five important reality - sim gaps

Domain	What the sim assumes	Reality
Contacts & friction	Foot touches ground with a fixed μ.	μ depends on moisture, wear
Actuators	Motor delivers exact commanded torque.	saturation, backlash, delay
Sensors	Ground-truth pose, no noise.	bias drift, dropped frames
Timing / latency	Command applied instantly.	10–20 ms of delay
Inertia & mass	Values copied from CAD.	cables, payload, tolerances

Domain randomization: training the controller under a wide range of parameters is the main tool used to survive the transfer.

When simulation does not help

Simulation is powerful, but not universal. There are questions it **cannot** answer properly.

CASE A
Fine tactile behaviour

Gripping a soft object, sliding fingers, deformable materials — physics engines still handle these poorly.

CASE B
Human interaction

Whether a passer-by will step aside for the robot cannot be inferred from a physics engine.

CASE C
Certification & safety proofs

Automotive and aviation safety cases require real-hardware data. Sim can support, not replace.

Simulation reduces the need for hardware experiments, but it never removes it.

Main components of robot simulations

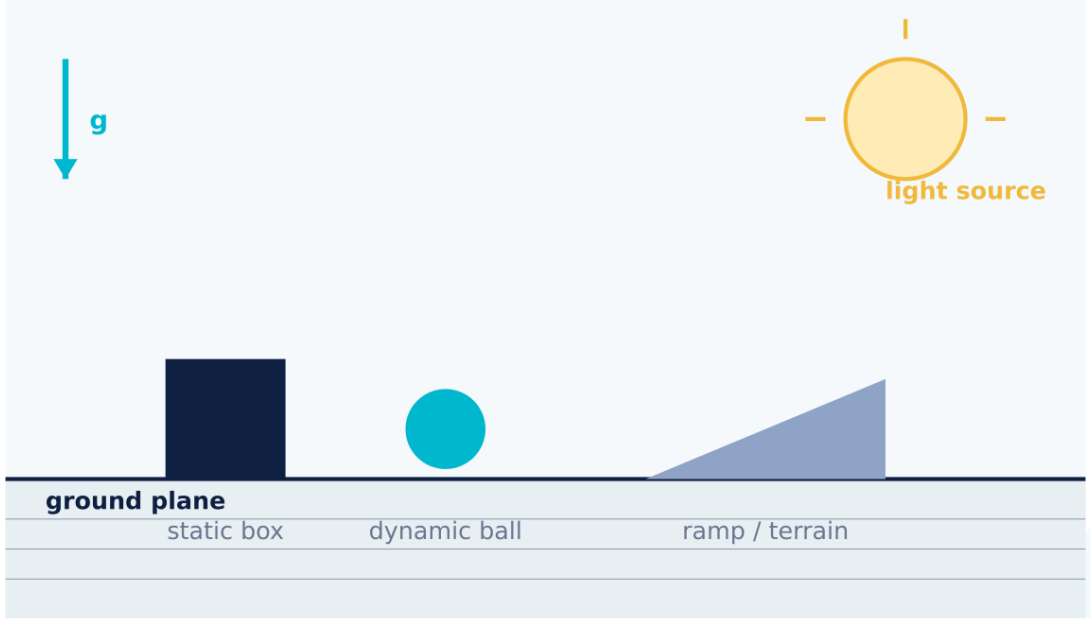
1	**World**	The stage: gravity, ground, obstacles, lighting.
2	**Robot model**	Links, joints, masses, inertias: the robot as data.
3	**Physics engine**	Solves Newton–Euler, steps time forward, handles contacts.
4	**Sensors**	Encoders, IMU, camera, LiDAR, force / torque.
5	**Actuators**	Position, velocity, torque commands.
6	**Closed loop**	State → controller → command → physics step → repeat.

The world

Before we have a robot, we need a world for it to stand in. In a simulator, **the "world"** is the coordinate frame in which everything else is placed, plus every object that is not part of the robot itself.

→ What lives inside the world (ground, obstacles, lighting)
→ Gravity, friction, and other global physical parameters
→ Only when the world is defined can we place a robot in it

Components of a simulation world



world coordinate frame

Five kinds of entity

- **Ground plane** — infinite flat surface the robot stands on
- **Static obstacles** — walls, tables, tools
- **Dynamic objects** — balls, boxes to be manipulated
- **Terrain** — heightfields, ramps, stairs
- **Lights** — needed only if cameras are simulated

Every entity has a pose (position + orientation) in the world frame, that is what "being in the world" means.

The world’s global parameters

Gravity

A constant force vector applied to every body in the world.

$$g = (0, 0, -9.81)\ m/s^2$$

Set to zero for underwater or space robots. Set to a custom value to test what happens on the Moon.

Friction coefficients

Assigned per surface pair, not globally.

- rubber-on-concrete $\mu \approx 0.9$
- rubber-on-wet-tile $\mu \approx 0.3$
- steel-on-steel $\mu \approx 0.6$
- plastic-on-plastic $\mu \approx 0.2$

Wrong μ is the single most common cause of "works in sim, fails on hardware".

The robot model: The robot as data

Inside the simulator the robot is a **tree of rigid bodies** connected by joints, each body carrying mass, inertia and geometry. Everything the physics engine ever sees flows from this description.

In the next six slides

1. Links — the rigid segments
2. Joints — how links connect
3. Joint types — hinge, slider, ball, free
4. Mass and inertia — the things that make the body *feel* physical
5. Collision vs. visual geometry — the two-mesh convention

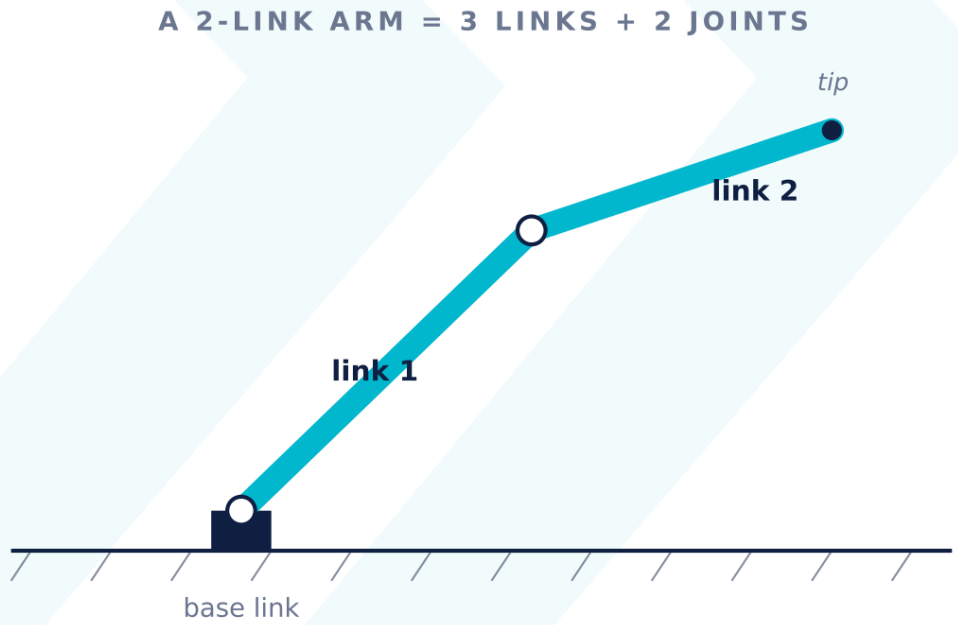
Links — the rigid segments

Definition

A **link** is a rigid body that does not deform. Its shape is fixed; only its position and orientation change over time.

Examples in a typical robot:

- Chassis of a car
- Individual leg segments — thigh, shin, foot
- Upper arm, forearm, hand palm
- Wheels, individual fingers



Joints — how links connect

Definition

A **joint** is a constraint between two links that removes some of their relative motion but allows the rest.

Every joint has:

- A **parent** and a **child** link
- A **type** (which motion it allows)
- An **axis** (around/along which motion happens)
- **Limits** — min/max angle or displacement

Degrees of freedom (DOF)

Each joint contributes 1 or more **DOF** — the count of independent numbers needed to describe the robot's pose.

- robot arm: 6 DOF
- 6 joints Quadruped ≈ 12 DOF
- Humanoid: 25–40 DOF
- High dexterity hand alone → 24 DOF

More DOFs = larger state vector = more expensive to simulate.

Four joint types

TYPE 1

Revolute (hinge)



1 DOF · rotates around an axis. Elbows, knees, wheels.

TYPE 2

Prismatic (slider)



1 DOF · slides along an axis. Linear actuators, telescopic arms.

TYPE 3

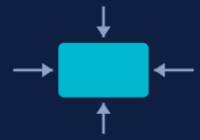
Ball (spherical)



3 DOF · rotates in all directions. Shoulder, hip joints.

TYPE 4

Free (floating base)



6 DOF · unconstrained. Used for the base of legged robots and drones.

Every humanoid or quadruped has exactly one "free" joint at the root. That is what makes it a floating-base robot.

Mass and inertia — how a link feels physical

Mass · Centre of mass

Mass (m) — total kilograms of the link.

Centre of mass (CoM) — the point where you could balance it on a fingertip.

> F = m · a (Newton's 2nd law)

Mass tells the engine how hard it is to accelerate. CoM tells it where the force effectively acts.

Inertia tensor

A **3×3 matrix** that describes how mass is spatially *distributed,* not just its value.

A pencil is easy to spin around its long axis, hard around the short one. That difference lives in the inertia tensor.

> **Practical rule —** get mass and inertia from your CAD tool.
> Never estimate them by hand: wrong inertia is the fastest way to make a sim explode.

Collision vs visual geometry: the two-mesh convention

Every link in a well-built model carries **two** geometries. If you use only one, either the physics is slow or the renderer looks bad.

Collision geometry

Simplified primitives: **boxes, spheres, capsules, convex hulls**. Contact detection runs on this. Keep triangle counts low: dozens, not thousands.

≈ 20–50 triangles per link

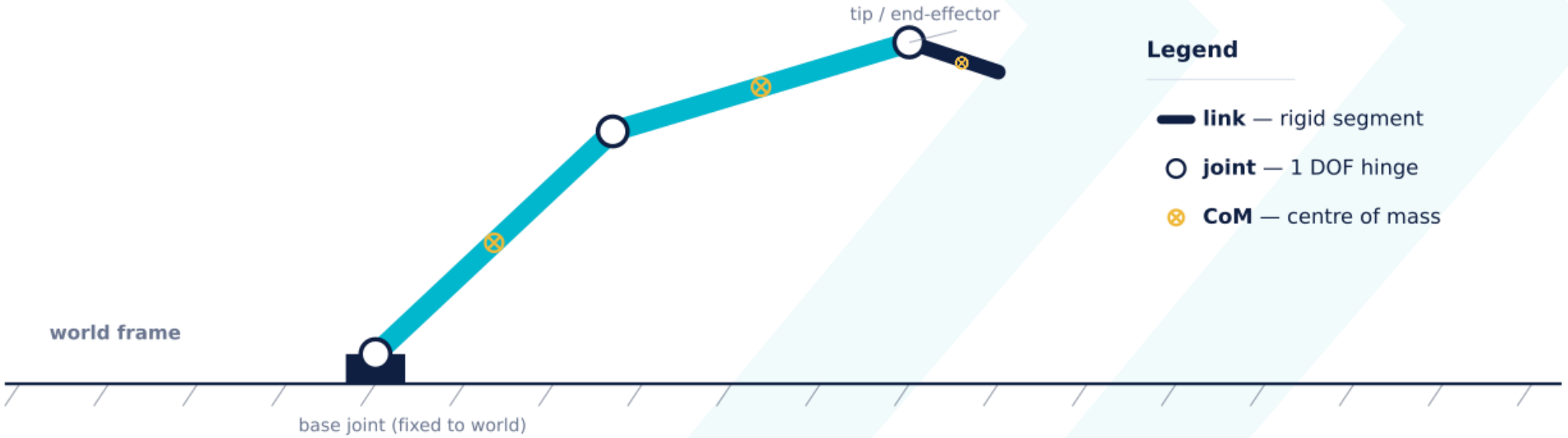
Visual geometry

The full CAD mesh. Used only by the renderer, never by physics. Can have **tens of thousands** of triangles without slowing the sim.

≈ 10 000+ triangles per link

Common mistake — using the visual mesh for collision. Sim drops from 1000 fps to 3 fps.

Example: Robot model



That is **everything** the physics engine needs to know about the robot.
Now we need a way to write it down.

How is the robot written down for the simulator?

A physics engine cannot read a CAD file. Between "the robot is designed in SolidWorks" and "the robot exists inside Gazebo/MuJoCo" sits a text file: an **XML description** of every link, joint, mass and geometry.

Three formats dominate the field

1. **URDF** Unified Robot Description Format — the ROS standard
2. **SDF** Simulation Description Format — Gazebo's native format
3. **MJCF** MuJoCo XML Configuration Format — physics-first

We describe each one in turn, then compare them side by side.

URDF – The ROS standard

What it is

Unified Robot Description Format. An XML file that describes **one** robot as a tree of <link> and <joint> nodes.

Strengths:
- universal in the ROS ecosystem
- tool-friendly (RViz, MoveIt)

Weaknesses:
- single robot only
- no world
- no closed kinematic loops
- weak sensor support

```
<robot name="arm">
  <link name="base">
    <inertial>
      <mass value="2.0"/>
    </inertial>
  </link>
  <joint name="j1" type="revolute">
    <parent link="base"/>
    <child link="link1"/>
    <axis xyz="0 0 1"/>
  </joint>
</robot>
```

Where you'll meet it — ROS 1 / ROS 2, MoveIt, RViz, and as the input to Gazebo when converted through urdf2sdf.

SDF – Simulation Description Format

What it is

URDF's more expressive cousin. One SDF file describes a whole world — multiple robots, static objects, lights, physics parameters, sensor definitions.

Strengths:

- multi-robot
- native sensor/plugin support
- closed kinematic chains
- world included

Weaknesses:

- less standardised outside the Gazebo ecosystem

```
<sdf version="1.9">
<world name="lab">
  <light .../>
  <model name="ur5">
    <link .../>
      <joint .../>
        <sensor type="camera" .../>
   </model>
   <model name="table">
       ...
    </model>
 </world>
</sdf>
```

Where you'll meet it — Gazebo Classic, Ignition Gazebo. When you write a Gazebo world file, you write SDF.

MJCF – Physics-first XML

What it is

MuJoCo XML Configuration Format: the native format for MuJoCo. Compact, actuator- and sensor-first, and centered around dynamics

Strengths:

- Concise
- Built-in actuators/sensors
- Defaults save huge amounts of boilerplate

Weaknesses:

- MuJoCo-only, less ecosystem tooling than URDF.

```
<mujoco model="arm">
  <worldbody>
    <body name="link1">
      <joint type="hinge" axis="0 0 1"/>
        <geom type="capsule" size="0.05 0.3"/>
    </body>
  </worldbody>
  <actuator>
    <motor joint="link1"/>
  </actuator>
</mujoco>
```

Where you'll meet it — MuJoCo, DeepMind Control Suite, Robotics benchmarks, most legged-robot projects.

Choosing between URDF, SDF, MJCF

CAPABILITY	URDF	SDF	MJCF
Whole world in one file	no	**yes**	**yes**
Multiple robots	no	**yes**	**yes**
Closed kinematic loops	no	**yes**	**yes**
Sensors as first-class tags	weak	**yes**	**yes**
Actuator model built in	weak	medium	**strong**
ROS ecosystem native	**yes**	yes (via Gazebo)	no
Compactness	verbose	verbose	**compact**

A single project often uses two — URDF for ROS control, and SDF or MJCF for the physics side.

The physics engine

Once the world and the robot model are described, the **physics engine** is the piece of software that takes them and computes how they move, millisecond by millisecond.

- → The equation of motion the engine solves
- → Numerical integration — from continuous to discrete time
- → The timestep Δt — the most important dial to turn
- → Constraints — joints, limits, closed loops
- → Contacts — the hardest part

The equation solved on every timestep

$$M(q)\ddot{q} + C(q,\dot{q}) = T + J^{T}\lambda$$

- **M** **Mass matrix** — how hard the robot resists acceleration
- **q** **Joint positions** — the DOF vector
- **$\dot{q}$, $\ddot{q}$** **Velocity + acceleration** of those joints
- **C** **Coriolis + gravity** — the "free" forces already acting
- **τ** **Actuator torques** — the controller's contribution
- **$J^{T}\lambda$** **Constraint / contact forces** — reaction from the world

Newton–Euler dynamics for articulated rigid bodies. Every step the engine solves this for $\ddot{q}$.

Numerical integration

The problem

The equation of motion tells us $\ddot{q}$. But we want to know q(t) — the position at every future time.

$$\dot{q} \leftarrow \dot{q} + \ddot{q} \cdot \Delta t$$
$$q \leftarrow q + \dot{q} \cdot \Delta t$$

One tiny step at a time **— this is semi-implicit Euler,** the default in most simulators.

Common integrators

Explicit Euler - fast, stable only at small Δt

Semi-implicit Euler - default, energy-preserving

Runge–Kutta 4 - accurate, ~4× slower per step

Implicit methods - stable at large Δt, expensive

MuJoCo, Gazebo, Isaac all default to semi-implicit Euler. It is the best value-per-CPU-cycle for robotics.

How to choose timestep

The timestep controls a three-way trade-off between **speed**, **accuracy**, and **stability**. Get it wrong and either your sim runs at 3 fps or the model explodes on the first contact.

TOO SMALL

Δt = 0.1 ms

Very accurate, but sim runs 10× slower than needed. Training takes 10× longer.

TYPICAL

Δt = 1–5 ms

Sweet spot for legged robots. Enough resolution for foot contacts, still fast.

TOO LARGE

Δt > 10 ms

Contacts explode. Robot tunnels through the floor. Numerical instability.

Constraints of a simulated body

A robot's parts are not free-flying, they are wired together by **constraints** the engine must respect on every step.

Joint constraints

A hinge allows exactly one axis of rotation. The engine keeps the child link from separating from the parent along the other axes.

Joint limits

Range of motion: a knee cannot bend backward. Once the angle hits the limit, a large force pushes back.

Closed loops

A four-bar linkage or a differential drive shares a link between two chains. URDF cannot express this; SDF/MJCF can.

Contact non-penetration

Two bodies cannot occupy the same space. The single hardest constraint — deserves its own slide.

Contacts: The most difficult part of a robot simulation

Why contacts are hard

Contact is **discontinuous** — a foot is either touching the floor or not, with nothing in between. The moment of impact is a step function in every equation, and step functions break most numerical methods.

For a walking robot, this happens **hundreds of times per second**.

What the engine must resolve

1. **Non-penetration** — foot cannot enter floor
2. **Normal force** — magnitude of the ground push-back
3. **Friction** — static, sliding, torsional (μ)
4. **Restitution** — how bouncy the collision is

> All at once, on every timestep, for every colliding pair.

Different engines take different approaches — this is where they visibly differ.

Sensors: The robot's way of perceiving the world

Physics runs the world; **sensors** are how the controller finds out what happened. Every sensor is a plugin that reads simulator state and produces a measurement — the same measurement the real hardware would produce.

Five sensor families

1. Joint encoders — the cheapest, most reliable sensor
2. IMU — inertial motion of the body frame
3. Camera — RGB and depth images
4. LiDAR — ray-cast point clouds
5. Force / torque — end-effector wrench

\+ Noise — the honest addition to every sim sensor

Joint encoders — every joint's own scale

What it measures

Position q and (optionally) velocity $\dot{q}$ at every joint. In simulation these are simply **read from the state vector** — no ray casting, no rendering, essentially free.

In hardware they come from optical or magnetic encoders — 0.01°–0.1° resolution is typical.

Typical properties

- Rate 500 – 2000 Hz
- Resolution 0.01 – 0.1 deg
- Latency ≈ 0 in sim; 1–2 ms real
- Noise very low

Uses

- Proprioception
- PID inner loop
- whole-body-control feedback.

The cheapest sensor to simulate; also the closest to reality out of the box.

IMUs – Inertial Measurement Units

What it measures

Linear acceleration (accelerometer) and **angular velocity** (gyroscope) of the body it is mounted on — expressed in the sensor frame.

> $a = (a_x, a_y, a_z)$ [m/s²] $\omega = (\omega_x, \omega_y, \omega_z)$ [rad/s]

Why it matters

Every legged robot has one — usually inside the torso. It provides the **orientation estimate** the balance controller needs.

- Rate: 100 – 1000 Hz
- Real IMUs have **bias drift**, temperature dependence
- Sim IMUs are noise-free — add noise or you fool yourself

Without an IMU, a biped cannot tell "up" from "down". Fusing IMU with encoders is the classic state-estimation problem.

Cameras

What it produces

A rendered **image** from a virtual pinhole camera. Comes in three flavours:

RGB colour image — matches a webcam

D depth image — one distance per pixel

SEG segmentation mask — object ID per pixel (ground-truth labels for training)

The cost of cameras

Cameras are the expensive sensor to simulate. Every frame requires a full render pass.

Physics-only sim 2000+ fps
+ 1 camera 640×480 ~ 200 fps
+ photorealistic ~ 60 fps

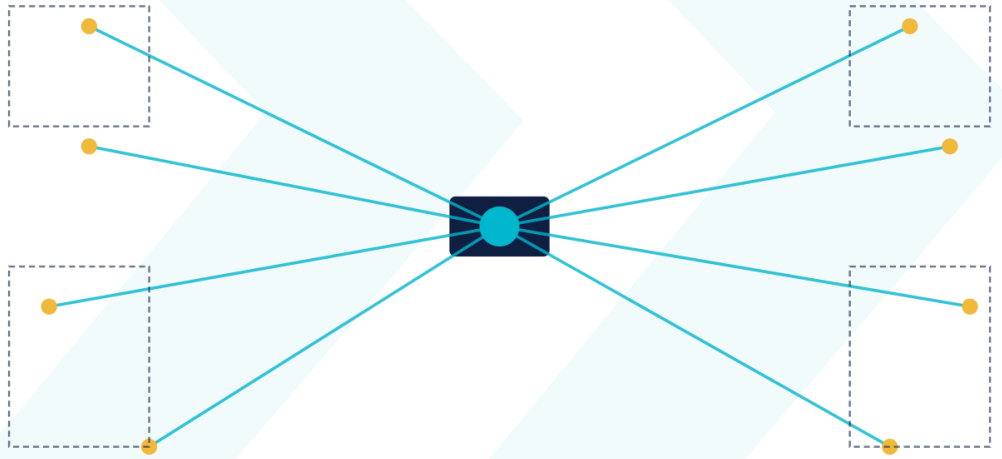
RL researchers usually train blind (no camera), then add vision at the end

LiDARs – geometry, not color

What it is

LiDAR (Light Detection And Ranging) fires laser rays out in a pattern and reports the distance to the nearest object each ray hits. The output is a **point cloud.**

In simulation each ray is a single ray-cast against collision geometry — much cheaper than a full render.



rays cast, distances returned

Common configurations — 2-D scan (mobile bases), 360° 3-D LiDAR (autonomous vehicles), 16- to 128-line rotating (Velodyne, Ouster).

Force / torque — feeling contact

What it measures

A 6-DOF **wrench:** three force components and three torque components — passing through a specific mount point.

$$F = (F_x, F_y, F_z)\ [N]^{\top} = (\tau_x, \tau_y, \tau_z)\ [N \cdot m]$$

Where you find them

- Under the foot — ground reaction force for legged robots
- At the wrist — for delicate manipulation
- Between two links — measure internal loads

In sim you get ground truth F/T for free; on hardware it is one of the most expensive and noisy sensors in the lab.

Noise – Sim sensors are too clean

By default, a simulated sensor returns **exact** values. Real sensors do not. If you tune your controller against a perfect measurement it will fail on the real robot. Fix this the honest way: inject noise into the sim.

TYPE 1

Gaussian noise

Zero-mean random offset added to each reading. Encoder ± 0.05°, IMU ± 0.02 m/s².

TYPE 2

Bias drift

A slowly-varying offset that survives filtering. IMU bias is why orientation estimates drift over minutes.

TYPE 3

Latency & dropped frames

Sensors arrive late, sometimes not at all. Introduce 10–30 ms delay to match real WiFi/USB stacks.

Rule — if your solution only works with noise = 0, it is not ready for hardware.

Actuators

Sensors give the controller information. **Actuators** are how the controller talks back — by applying forces or torques at specific joints. Every simulator supports several "control modes", each with different underlying physics.

Three control modes and the limits behind them

1. Position control — the safest, easiest to use
2. Velocity control — for wheels and mobile bases
3. Torque control — what modern legged robotics needs

\+ Motor limits — the constraints beginners forget

Position control

What you write

You give the simulator a target angle qdes. Inside, an internal **PD servo** converts that into a torque:

$$\tau = K_p(q_{des} - q) - K_d\dot{q}$$

When to use it

- ✓ Industrial 6-DOF arms following a trajectory
- ✓ Pick-and-place, waypoint following
- ✓ Getting a demo working *fast*

- ✗ Contact-rich manipulation
- ✗ Legged locomotion (too stiff on contact)

K_p = stiffness, K_d = damping. The simulator picks the torque; you do not see it.

Velocity control

What you write

You give the simulator a target rate $\dot{q}_{des}$. An internal **velocity servo** drives velocity error to zero:

$$\tau = K_v(q_{des} - q)$$

Canonical use case

Wheels. A differential-drive robot receives two velocity commands — left wheel rate, right wheel rate — and rolls.

```
left_wheel.set_vel( 4.0) # rad/s
right_wheel.set_vel(4.0) # → straight
left_wheel.set_vel(2.0) right_wheel.set_vel(6.0) # → turn left
```

Also common for continuous rotation joints in a UR5 or arm-mounted spinning drill.

Torque control

What you write

You give the simulator the exact torque τ in newton-metres. **No inner loop, no hiding.** The dynamics unfold directly from what you commanded.

> τ = your value, applied as-is

Why legged robotics needs it

A rigid position servo cannot back off when the foot hits an unexpected obstacle. A torque-controlled leg can be **compliant** — soft on impact, stiff during stance.

Modern RL policies always output torques (or PD targets converted to torques). ANYmal, Spot, Unitree Go2, Tesla Optimus — all torque-controlled.

The trade-off: torque control makes the sim honest about your controller's mistakes — and there are always mistakes.

Motor limits

By default, a simulator will deliver **800 N·m** to a motor whose real spec is **40 N·m**. Your controller works in sim; on hardware it saturates in 100 ms and the robot falls over.

LIMIT 1
Torque

$\tau \in [-\tau_{max}, +\tau_{max}]$

Set from the motor datasheet. Clip the command; do not simulate infinite torque.

LIMIT 2
Velocity

$|\dot{q}| \leq \dot{q}_{max}$

A gearbox has a maximum output speed; commanding faster is physically meaningless.

LIMIT 3
Joint range

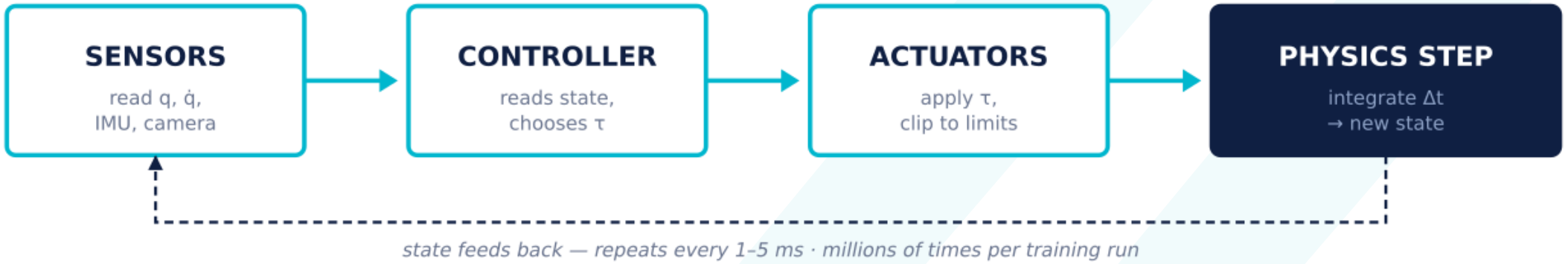
$q \in [q_{min}, +q_{max}]$

Physical stops — knees cannot hyperextend, elbows have a range. Enforce these.

In MJCF: <motor forcerange="..." ctrlrange="..." />

The closed loop

Now that we have world + robot + physics + sensors + actuators, we can wire them together. This is the loop that runs on every simulator, whether it is Gazebo, MuJoCo, or Isaac Sim.



Every robot simulator you will ever use is a different implementation of this loop.

Five simulators worth knowing about

Robot simulation is a crowded field. These are the most used robot simulators:

1	**Gazebo**	The ROS default. Classic + new (Ignition) branches.
2	**Webots**	Open-source, teaching-friendly.
3	**CoppeliaSim**	Modular, scripting-heavy, popular in research.
4	**NVIDIA Isaac Sim**	GPU-parallel, photorealistic, thousands of envs at once.
5	**MuJoCo**	Contact-first, control-first.

Gazebo Classic

The default simulator of ROS 1 and early ROS 2. If a robotics paper before 2022 shows a green ground plane and a robot named "PR2" or "TurtleBot", the screenshot is from Gazebo Classic. It is the tool everyone in industry **knows**, whether they still use it or not.

Strengths

Massive model library, ROS-native, mature plugins for sensors and controllers, wide community support, thousands of tutorials.

Weaknesses

End-of-life since 2023, aging renderer, contacts are less accurate than modern engines, single-threaded physics.

Under the hood of Gazebo Classic

Physics backend	
World format	SDF (.world). Robots authored in URDF are converted internally.
ROS interface	Via gazebo_ros_pkgs — a bridge that publishes joint states, sensors, images to standard ROS topics.
Rendering	OGRE 1 — functional but dated. Shadows and lighting look 2010-era.
License & cost	Apache 2.0 — free, open-source.

Status — "Gazebo Classic 11" is the final release. New projects should adopt New Gazebo (next slide) instead.

Ignition Gazebo

Around 2019 the Gazebo team started over. New codebase, modular libraries, new renderer, better physics integration. First called "Ignition", now simply **Gazebo**. Releases are named after cities: Citadel, Fortress, Garden, Harmonic, Ionic.

What changed vs Classic

- **Modular** — separate libraries (gz-physics, gz-rendering, gz-sim)
- **OGRE 2 / OptiX renderer** — much better lighting, shadows, PBR materials
- **Better distributed simulation** — multi-machine setups supported
- **Native ROS 2 bridge** — no more converter hacks

Physics engines, scene graph, sensors, actuators

Physics engines

DART — high-accuracy dynamics, humanoid-ready.
Bullet — fast, collision-focused.
ODE — legacy, still available.
All swappable at runtime.

Scene graph

Hierarchical entity system. Every world element (robot, sensor, static object) is a node with a pose. Add / remove / re-parent at runtime.

Sensor models

Camera (mono & stereo), depth camera, GPU-LiDAR, 3-D LiDAR, IMU, magnetometer, GPS/GNSS, contact, thermal. All configured via SDF plugins.

Actuators & locomotion

Joint control via plugins. Prebuilt **Ackermann steering** and **differential drive** plugins ship with the release.

Choosing the right Gazebo / ROS 2 pair

New Gazebo and ROS 2 are released independently, so **compatibility is version-dependent**.

ROS 2 RELEASE	FORTRESS	GARDEN	HARMONIC	IONIC
Humble (LTS)	✓ **recommended**	experimental	no	no
Iron	✓	✓	no	no
Jazzy (LTS)	✓	✓	✓ **recommended**	experimental
Rolling	—	—	✓	✓ **latest**

Bridge package: ros_gz. Configure topic-by-topic — every ROS topic maps to a Gazebo topic explicitly.

Webots simulator

Developed since 1998 by Cyberbotics, open-sourced in 2018. Ships as a **single installer** with graphical scene editor, sensor library, and 100+ example robots (e-puck, Nao, Atlas, Pioneer, TurtleBot). The path of least resistance for a first robotics course.

Strengths

Cross-platform installer, drag-and-drop scene editor, bindings for C, C++, Python, Java, MATLAB, ROS, ROS 2. Excellent didactic material.

Weaknesses

Custom scene format (.wbt — VRML-based, not URDF), smaller research community, weaker contact solver than DART or MuJoCo.

Under the hood of Webots

Physics backend	**ODE** — the classic Open Dynamics Engine.
Scene format	Custom .wbt (extended VRML). URDF and Xacro can be imported via converter.
Controller languages	C, C++, Python, Java, MATLAB, ROS 1, ROS 2 — pick any.
Rendering	OpenGL — modern, PBR materials, decent shadows. Not photorealistic.
Robot library	100+ ready-to-drive robots including e-puck, Nao, Atlas, Universal Robots, Boston Dynamics Spot.
License & cost	Apache 2.0 since 2018 — free, open-source.

Best fit — undergraduate labs, robotics competitions, quick prototyping without ROS.

CoppeliaSim – modular, scripting-heavy

Formerly V-REP, developed by Coppelia Robotics since 2006. Distinctive feature: **each object in the scene can carry its own script**. A robot arm, a conveyor belt, and a workpiece each run independent behaviour — a natural fit for factory-floor simulations.

Strengths

Per-object scripting, four swappable physics engines, remote APIs (Python, MATLAB, C++, ROS). Popular for research and industrial demos.

Weaknesses

Educational license is free, but full commercial license is paid. Learning curve steeper than Webots. Smaller community than Gazebo.

Under the hood of CoppeliaSim

Physics backends	Bullet, ODE, Vortex Studio, Newton. Switch mid-project without changing the scene.
Scripting	Lua (embedded), Python (add-on), C++. Threaded and non-threaded scripts per object.
Scene format	Native .ttt binary. URDF and Collada import supported.
Robot library	Comprehensive — industrial arms (UR, Kuka, Franka), mobile robots, humanoids, drones.
External APIs	ZeroMQ, WebSocket, ROS 1 & 2 — control from any host language.
License	Free for education. Commercial licences are paid.

Best fit — industrial workcell simulation, applications with many independent moving parts.

NVIDIA Isaac Sim — the GPU option

Built on NVIDIA's Omniverse platform. What sets it apart is not "another physics engine" — it is **GPU-parallel physics**. Thousands of independent robots step forward at the same time on one GPU. This is what makes modern robot RL feasible.

Strengths

4 000+ parallel envs on a single GPU, photorealistic rendering (RTX ray tracing), synthetic data generation, ROS 2 bridge, Isaac Lab for RL training.

Weaknesses

Requires an NVIDIA GPU (RTX class). Installation ~30 GB. Steep first-hour learning curve. Runs only on Linux and Windows.

Parallel physics on a GPU

The trick

NVIDIA's **PhysX 5** engine runs on the GPU. Instead of stepping one robot per timestep, it steps thousands of independent robots simultaneously — using the GPU's parallelism the same way a game renders millions of triangles.

This turns reinforcement learning from a multi-day server job into a **multi-hour workstation job**.

Concrete numbers

Gazebo 1 robot at 1× real-time
MuJoCo (CPU) 1 robot at 100× real-time
Isaac (1 GPU) 4096 robots × 100× real-time

Note — MuJoCo now has a GPU/JAX backend too (MJX). The gap has narrowed since 2023.

Photorealistic rendering + synthetic data

Isaac Sim uses NVIDIA's **RTX ray-tracing** renderer, the same one behind Omniverse and film-grade tools. This matters when you train perception — the images the network sees during training look close enough to hardware camera images that transfer works.

Practical implications

- **Synthetic dataset generation** — millions of labelled images without real cameras
- **Domain randomization** — vary lighting, textures, camera pose in one loop
- **Digital twin** — a photorealistic replica of a real factory
- **Isaac Lab** — RL training framework built on top, with legged/manipulation tasks preset

Best fit — vision-in-the-loop RL, autonomous driving, digital twins, industrial-scale training runs.

MuJoCo – contacts done right

Multi-Joint dynamics with Contact. Originally written by **Emo Todorov** at the University of Washington, purchased by DeepMind in 2021, open-sourced in 2022. The de-facto standard in control theory, biomechanics and legged-robot RL.

Strengths

Fast, stable contact solver. Analytic derivatives (for MPC and gradient-based control). Compact MJCF format. First-class Python bindings. Free and open-source.

Weaknesses

Bare-bones renderer (simulator, not visualiser). No native ROS. World library smaller than Gazebo. Not aimed at photorealism.

Why the field converged on MuJoCo

R1

Contact accuracy at speed

Convex, soft-constraint solver — 30+ contacts per timestep, no explosions.

R2

Generalized coordinates

One number per DOF — no cartesian drift.
Correct joint constraints for free.

R3

Everyone uses it

DeepMind Control, OpenAI Gym, Isaac Lab, robotics papers, biomechanics labs.

"

MuJoCo is designed from the ground up for the purpose of model-based optimization, and in particular optimization through contacts.

— MuJoCo authors
Which is exactly what legged robotics needs.

All five simulators side by side

ASPECT	GAZEBO (NEW)	WEBOTS	COPPELIASIM	ISAAC SIM	MUJOCO
Physics engine	DART / Bullet / ODE	ODE	4 backends	PhysX 5	MuJoCo (own)
GPU parallel	no	no	no	**yes**	via MJX
Photorealism	good	medium	medium	**excellent**	basic
Contact accuracy	good	medium	good	good	**excellent**
Model format	SDF / URDF	WBT / URDF	TTT / URDF	USD / URDF	MJCF / URDF
ROS support	**native**	bridge	bridge	bridge	via wrapper
License	**Apache 2.0**	**Apache 2.0**	EDU free / paid	NVIDIA EULA	**Apache 2.0**
Best fit	*ROS 2 mobile robots*	*teaching, competitions*	*industrial cells*	*vision + RL at scale*	*legged / control theory*

Which one should you use?

Your first robotics course	→ **Webots** — installer, examples, gentle learning curve.
You are building on ROS 2	→ **New Gazebo** — matched to your ROS release (Harmonic + Jazzy is the sweet spot).
You are training a legged robot policy	→ **MuJoCo (MJX)** or **Isaac Sim** — GPU-parallel physics is not optional at scale.
You need photorealistic images	→ **Isaac Sim** — the RTX renderer is unmatched.
You are writing a control-theory paper	→ **MuJoCo** — analytic derivatives, exact contact solver, tiny reproducer files.

Usually there is no one-size-fits-all solution

XML → mjModel → mjData — the three stages



Every MuJoCo program follows this exact shape. Write once, compile once (fast), step millions of times.

mjModel is static, mjData is alive

mjModel

Geometry, masses, inertia matrices, joint limits, actuator gains, sensor definitions. **Built once from the XML, never mutated.**

```
model.nbody # number of bodies
model.njnt # number of joints
model.body_mass # mass per body
model.jnt_range # joint limits
```

mjData

Everything that changes each step. Positions, velocities, applied controls, computed contacts, sensor readings. **Advanced by every mj_step() call.**

```
data.qpos # joint positions
data.qvel # joint velocities
data.ctrl # actuator commands
data.contact # active contacts
data.sensordata# sensor readings
```

To reset a simulation, you simply zero out mjData; mjModel is untouched.

Six tags build any MuJoCo model

```
<mujoco><worldbody>← the scene root
<geom type="plane" size="5 5 .1"/>← ground
<body name="link1">← rigid body
<joint type="hinge" axis="0 0 1"/>← 1 DOF
<geom type="capsule" size="0.05 0.3"/></body></worldbody>
<actuator>← how we drive it
<motor joint="link1" gear="10"/></actuator>
<sensor>← what we read
<jointpos joint="link1"/></sensor></mujoco>
```

The vocabulary

<worldbody>	the scene root
<body>	one rigid body
<joint>	how it connects to parent
<geom>	shape (visual + collision)
<actuator>	apply force at a joint
<sensor>	what the controller reads

<worldbody> and <geom>

The scene root

<worldbody> is the top of the tree. Anything inside it exists in world coordinates. Static objects — like the ground plane — sit directly here; robots go inside as nested <body> tags.

<geom> tags inside the worldbody create shapes: planes, boxes, spheres, capsules, meshes.

```
<worldbody>
<!-- infinite floor -->
<geom type="plane" size="5 5 .1"/>
<!-- static wall -->
<geom type="box" pos="2 0 1" size="0.1 2 1"/>
<!-- ball to knock over -->
<geom type="sphere" pos="0 0 0.2" size="0.1"/>
</worldbody>
```

<geom> tags outside a <body> are static; inside a <body> they move with it.

<body> and <joint>

Bodies nest into a tree

A **<body>** creates a rigid segment. Nesting one inside another makes the outer the parent, the inner the child.

The **<joint>** tag inside a body defines how it can move relative to its parent.

No joint → welded to parent.
Multiple joints → composite DOF (a 3-DOF wrist).

```
<body name="upper_arm" pos="0 0 0.6">
<joint name="shoulder" type="hinge" axis="0 1 0" range="-1.5 1.5"/>
<geom type="capsule" fromto="0 0 0 0 0 -0.3" size="0.04" mass="1.2"/>
<body name="forearm" pos="0 0 -0.3">
<joint name="elbow" type="hinge" axis="0 1 0"/> ... </body></body>
```

<actuator> and <sensor>

Actuators — how you push the world

```
<actuator>
<!-- torque motor -->
<motor joint="elbow" ctrlrange="-30 30" gear="1"/>
<!-- PD position servo -->
<position joint="shoulder"
kp="400" kv="40" ctrlrange="-1.5 1.5"/>
</actuator>
```

Sensors — what the controller sees

```
<sensor>
<jointpos joint="shoulder"/>
<jointvel joint="shoulder"/>
<framequat objtype="body" objname="torso"/>
<accelerometer site="imu_site"/>
<force site="foot_site"/>
</sensor>
```

Note the symmetry —<actuator> writes into data.ctrl, <sensor> reads from data.sensordata.

How MuJoCo handles contacts differently

Most engines treat contacts as **hard constraints** — a body cannot move by even 1 μm into another. MuJoCo introduces a small, controllable **softness** — bodies are allowed to overlap slightly, and the solver becomes a well-behaved convex optimization.

Consequence 1
Always converges

The convex solver has one unique solution. It never gets stuck, never explodes on stacked contacts.

Consequence 2
Differentiable

Because the constraint is smooth, you can compute $\partial \dot{q}/\partial \tau$ — required for MPC and policy gradients.

This is the technical reason legged-robot researchers migrated to MuJoCo.
The contact solver behaves nicely under optimisation.

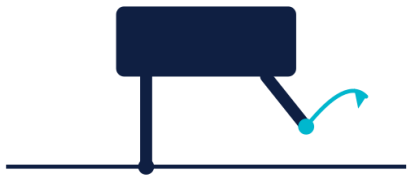
Why legged locomotion is harder than wheeled

WHEELED



continuous contact
stable, smooth dynamics

LEGGED



intermittent contact
hybrid, discontinuous

Four sources of pain

Intermittent contacts. Feet leave and strike the ground — dynamics become *hybrid* and non-smooth.

Balance. Support polygon is small; centre of mass leaving it means falling.

Underactuation. More DOFs than actuators — the base cannot be commanded directly.

Terrain. Ground shape, slope, friction all vary — sim must handle every case.

Three families of legged robots



FAMILY 1

Quadrupeds

4 legs, 12–18 DOF. Wide support polygon. The tractable entry point — where the field succeeded first.



FAMILY 2

Bipeds

2 legs, 6–12 DOF. Inherently unstable — balance is not optional.



FAMILY 3

Humanoids

Biped + arms + head. 25–40 DOF. The field's current frontier.

Difficulty grows non-linearly with each leg removed and each arm added.

Quadrupeds

Why they were the first success

Four feet form a wide **support polygon**. Even standing still, the centre of mass has ample room to shift before the robot tips. During walking, three feet are usually on the ground — *statically stable* gaits are possible.

Simulation load: 12–18 DOF, ~1 ms timestep,
500–1000 fps on a laptop.

- **Foot friction** — controls slip during turns
- **Contact scheduling** — trot, walk, gallop patterns
- **Terrain** — heightfields, slopes, stairs
- **Actuator saturation** — knees hit torque limits

Bipeds

Why bipedal is a step change

A biped has at most two feet on the ground — often just one during a step. The support polygon shrinks to a single foot-print. **Walking becomes a controlled fall** — every step, the CoM leaves the polygon and must be caught by the next footstep.

Simulation load: 6–12 DOF for legs alone.
Timestep 1 ms is the norm.

Real platforms

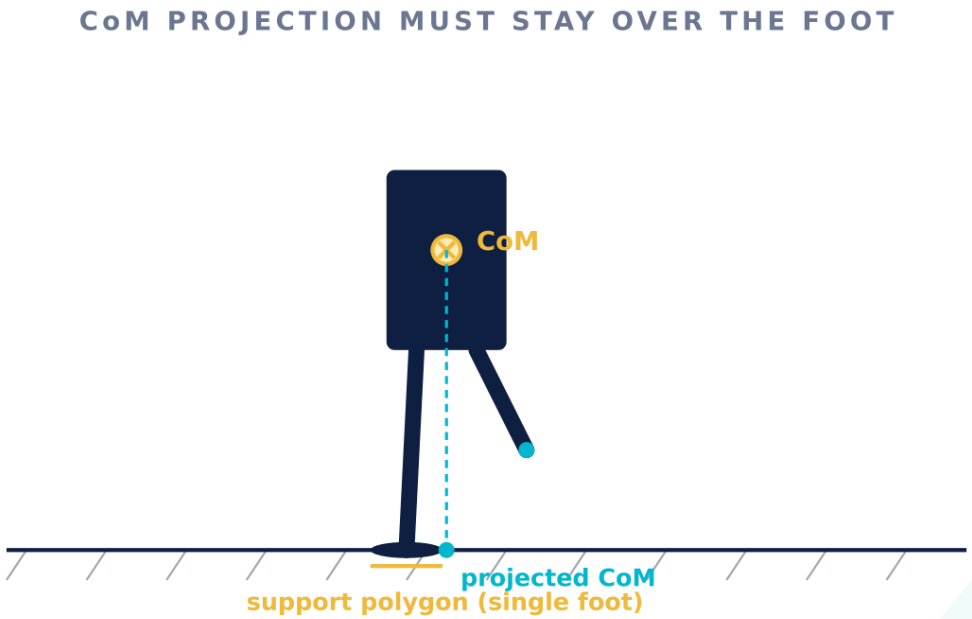
Agility Robotics Cassie — spring-loaded legs, no torso. Research favourite for locomotion.

Rabbit — the classical five-link planar biped used in most control-theory textbooks.

Compass gait — the simplest walker; two legs, no knees. Common in coursework.

Bipeds without a torso are called "legs-only" — used to isolate the locomotion problem.

Balance



Three ideas keep bipeds upright

Zero Moment Point (ZMP) — a virtual point on the ground where the moment integral is zero. Keep ZMP inside the support polygon.

Whole-body angular momentum — arms and torso swing to shift momentum without moving CoM.

Humanoids — biped plus a whole upper body

Anatomy

Two legs + torso + two arms + head, sometimes with a dexterous hand. **25–40 DOF** is typical; a robot with two 5-finger hands crosses **70 DOF**.

Simulation load: full model at 1 ms timestep runs at 400–800 fps on a single core, ~50 000 fps on MJX with a decent GPU.

Real platforms in 2025

Boston Dynamics Atlas — hydraulic, then electric. The gold standard for athletic humanoids.

Agility Digit — commercial warehouse humanoid.

Unitree H1 / G1 — affordable research humanoids.

Tesla Optimus — high-visibility commercial push.

Figure 02, 1X Neo — startup entrants.

Whole-body control — every joint couples to every other

A quadruped's legs are mostly independent. A humanoid's arms and legs **share the torso** — moving one arm shifts the entire CoM, which changes what the legs must do. The controller has to reason about every joint at once.

TASK 1
Balance
Keep the CoM projection over the support foot — always.

TASK 2
Locomotion
Move the base at a commanded velocity, in a stable gait pattern.

TASK 3
Manipulation
Track an end-effector target — arms doing useful work — *while* balance + locomotion continue.

Modern humanoids solve all three simultaneously through a single QP or neural policy.
This is what "whole-body control" means.

How do you know your simulation is correct?

Four checks — the earlier in the project, the better.

Free-fall test	Drop a body of known mass and see if it hits the ground at the expected time. Catches wrong gravity, wrong timestep.
Pendulum swing	Release from a known angle, measure the period. Catches wrong inertias.
Static stance	Put the robot on the ground with zero torque. Should stand — briefly, if unstable, indefinitely if statically stable. Catches mass and CoM errors.
Real-hardware log	Play back an actual torque trajectory recorded from hardware; compare positions. Catches every remaining gap.

Five possible issues

Symptom	Likely cause	Fix
Robot flies away or explodes	Wrong inertias link exported from CAD with 0 or absurd values.	Recompute I from mass + geometry
Sim runs at 3 fps	Collision meshes are the full visual CAD — thousands of triangles per foot.	Use boxes / spheres / capsules
Robot slides on the floor	Friction coefficient defaulted to a low value.	Measure it or tune to 0.8–1.0
Solver diverges at contacts	Timestep too large for stiff contact / high mass ratio.	Drop Δt from 5 ms → 1 ms
Works in sim, fails on hardware	Commanded torques exceed physical motor limits — sim happily obeyed.	Set ctrlrange & forcerange

Conclusion

1 **Simulation is a controlled approximation.**

Useful *because* it drops details, never mistake it for reality.

2 **Model quality matters more than solver.**

Wrong inertias, wrong friction, wrong limits → wrong answer, however fast the engine runs.

3 **Contacts dominate legged robotics.**

Everything else is easy compared to modelling a foot touching a floor.

4 **Always find the best simulator for the task**

MuJoCo for humanoids: Fast, open-source, and standard across academia, RL and biomechanics.

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

Thank you for your attention!

From physics engines to humanoid robots

Rudolf Krecht
krecht.rudolf@ga.sze.hu

JÁRMŰIPARI KUTATÓKÖZPONT
VEHICLE INDUSTRY RESEARCH CENTER